

Escola Profissional Bento de Jesus Caraça

Delegação de Lisboa

Sistema de Gestão de Hotel

Relatório do Trabalho — Módulo 11

Disciplina: PSI — Programação de Sistemas de Informação

Curso: GPSI

Turma: 12ºGPSI

Ano letivo: 2026/2027

Alunos:

Gonçalo Romão

Diogo Gonçalves

Cristóvão Barros

Martim Ogando

Professor: Breno Sousa

Índice

1. Introdução
2. Objetivos do Trabalho
3. Análise do Problema
4. Estrutura do Projeto
5. Classes do Projeto
 - 5.1 Pessoas (classe abstrata)
 - 5.2 Hospede
 - 5.3 Funcionario
 - 5.4 Gerente
 - 5.5 Recepcionista
 - 5.6 TecnicoManutencao
 - 5.7 TecnicoRecepcao
 - 5.8 Quartos (classe abstrata), QuartoSimples e QuartoLuxo
6. Herança
 - 6.1 Herança simples
 - 6.2 Herança múltipla
7. Classe Abstrata
8. Polimorfismo
9. Propriedades, Getters e Setters
10. Estruturas de Dados e Ciclos
11. Fluxograma
12. Funcionamento do Programa
13. Testes Realizados
14. Dificuldades Encontradas
15. Conceitos de Programação Aplicados
16. Conclusão
17. Verificação Final

Índice de Imagens

Figura 1 — Estrutura geral das classes

Figura 2 — Estrutura das classes de Quarto

Figura 3 — Fluxograma geral: reserva, estadia e check-out

Figura 4 — Fluxograma: fluxo completo com recepção e gerência

Figura 5 — Fluxograma: fluxo específico do QuartoLuxo

1. Introdução

Este relatório descreve o trabalho desenvolvido no âmbito do Módulo 11 da disciplina de PSI, do curso GPSI. O objetivo do trabalho era desenvolver, em Python, um sistema de gestão para um hotel, capaz de gerir quartos de diferentes tipos (simples e luxo) e as reservas feitas pelos hóspedes.

O enunciado pedia explicitamente que o projeto demonstrasse o uso de vários conceitos de Programação Orientada a Objetos: herança simples, herança múltipla, polimorfismo, classes abstratas e organização do código em módulos. Também era obrigatório usar estruturas de dados (como listas) e ciclos (for/while) sempre que necessário.

O projeto foi desenvolvido inteiramente em Python, com o código dividido em vários ficheiros e pastas (módulos), de forma a separar as responsabilidades: uma pasta para as pessoas (hóspedes e funcionários), outra para os quartos, e um ficheiro principal (main.py) que junta tudo e corre o programa através de um menu interativo no terminal.

2. Objetivos do Trabalho

Segundo o enunciado, a equipa foi "contratada" para desenvolver, para um hotel, um sistema que permitisse gerir:

- Quartos de diferentes tipos (simples, luxo);
- Reservas feitas por hóspedes.

Além disto, o enunciado exigia que o sistema fosse construído com recurso a herança simples, herança múltipla, polimorfismo, classes abstratas e módulos, usando estruturas de dados e ciclos sempre que fosse necessário.

Comparando com o que foi realmente implementado, o projeto cumpre estes objetivos: existe uma classe abstrata `Pessoas` da qual derivam `Hospede` e `Funcionario` (e, a partir desta, `Gerente`, `Recepcionista` e `TecnicoManutencao`); existe uma classe abstrata `Quartos` da qual derivam `QuartoSimples` e `QuartoLuxo`; existe herança múltipla na classe `TecnicoRecepcao`; e existe polimorfismo no método `mostrar_informações()`, implementado de forma diferente em cada subclasse. O código está também organizado em módulos (pastas `Pessoas`, `Pessoas/Funcionarios`, `Pessoas/Hospedes` e `Quartos`), e `main.py` junta tudo através de imports.

3. Análise do Problema

O problema a resolver é a gestão básica de um hotel: é preciso saber que quartos existem, quais estão ocupados, quem são os hóspedes e quanto cada um deve pagar pela estadia, e também gerir os funcionários do hotel (gerente, rececionista e técnicos de manutenção).

O programa desenvolvido resolve isto através de duas listas principais, criadas no início do `main.py`: `lista_quartos` (com os quartos disponíveis no hotel) e `lista_hospedes` (inicialmente vazia, que vai sendo preenchida à medida que os hóspedes fazem check-in). O utilizador interage com o sistema através de um menu de texto no terminal, podendo entrar como "hóspede" (para reservar um quarto, consultar a conta ou fazer check-out) ou como "funcionário" (para listar hóspedes, listar quartos, gerar relatórios de funcionários ou registar reparações).

4. Estrutura do Projeto

O projeto está organizado em módulos (pastas), separando pessoas de quartos, e dentro das pessoas, separando hóspedes de funcionários. A tabela seguinte resume os ficheiros existentes:

Ficheiro	Função
main.py	Ficheiro principal. Cria os quartos e os funcionários, e corre o menu interativo do programa.
Pessoas/Pessoa.py	Define a classe abstrata Pessoas, base de todas as pessoas do sistema.
Pessoas/Hospedes/Hospedes.py	Define a classe Hospede, que herda de Pessoas.
Pessoas/Funcionarios/Funcionario.py	Define a classe Funcionario, que herda de Pessoas.
Pessoas/Funcionarios/Gerente.py	Define a classe Gerente, que herda de Funcionario.
Pessoas/Funcionarios/Recepcionista.py	Define a classe Recepcionista, que herda de Funcionario.
Pessoas/Funcionarios/TecnicoManutencao.py	Define a classe TecnicoManutencao, que herda de Funcionario.
Pessoas/Funcionarios/TecnicoRecepcao.py	Define a classe TecnicoRecepcao, que herda de Recepcionista e de TecnicoManutencao (herança múltipla).
Quartos/Quartos.py	Define a classe abstrata Quartos e as subclasses QuartoSimples e QuartoLuxo.

Esta organização em pastas (Pessoas, Pessoas/Hospedes, Pessoas/Funcionarios e Quartos) é precisamente o que o enunciado pedia quando exigia o uso de módulos.

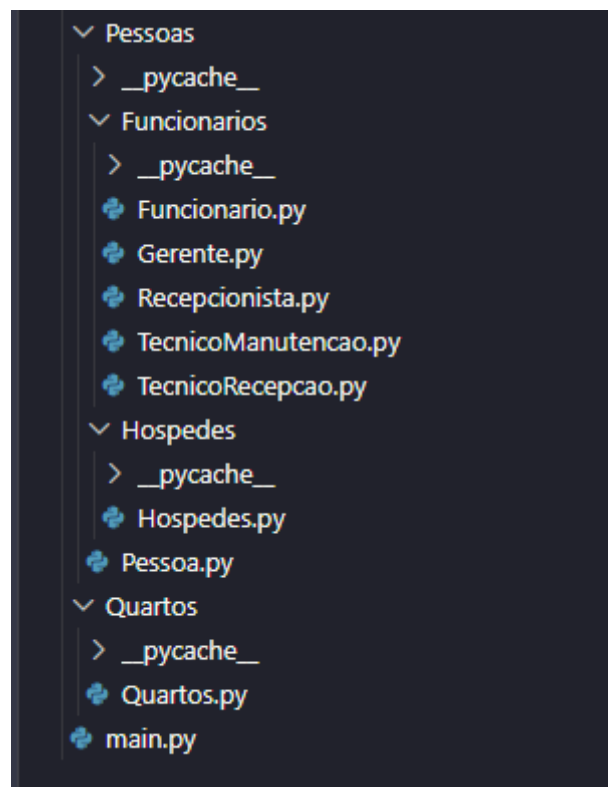


Figura 1 — Estrutura geral das classes

5. Classes do Projeto

Esta secção descreve, uma a uma, as classes realmente existentes no código entregue.

5.1 Pessoas (classe abstrata)

Herda de: ABC (classe abstrata do módulo abc)

Principais atributos:

- nome (propriedade, com validação no setter — não pode ser vazio)
- idade (propriedade, com validação no setter — não pode ser negativa)

Principais métodos:

- `__init__(self, nome, idade)`
- `mostrar_informações(self)` — método abstrato (`@abstractmethod`), sem implementação nesta classe

Como é utilizada no programa:

Pessoas é a classe base de todo o sistema. Não pode ser instanciada diretamente (por ser abstrata), servindo apenas de "molde" para `Hospede` e `Funcionario`, que são obrigadas a implementar o método `mostrar_informações()`.

```
class Pessoas(ABC):
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    @abstractmethod
    def mostrar_informações(self) -> None:
        pass
```

5.2 Hospede

Herda de: Pessoas

Principais atributos:

- `__dias_estadia` (privado)
- `__quarto` (privado — propriedade quarto, só aceita um quarto que esteja livre)

Principais métodos:

- `atribuir_quarto(self, quarto)`
- `calcular_conta(self)` — devolve o preço total da estadia, calculado pelo quarto
- `mostrar_informações(self)` — mostra nome, idade e número do quarto
- `checkout(self)` — liberta o quarto e limpa a associação

Como é utilizada no programa:

É criado um objeto `Hospede` quando alguém faz check-in através do menu (opção "1 - Fazer reserva"). O quarto só é atribuído se estiver livre, graças à validação feita no setter da propriedade `quarto`.

5.3 Funcionario

Herda de: Pessoas

Principais atributos:

- `_id_func` (protegido)
- `_salario` (protegido, comentado no código como "atributo protegido")

Principais métodos:

- `mostrar_informações(self)` — mostra nome, id, idade e salário

Como é utilizada no programa:

Funcionario é a classe-base comum a todos os trabalhadores do hotel (Gerente, Recepcionista, TecnicoManutencao e, indiretamente, TecnicoRecepcao). No `main.py` são criados um gerente, um rececionista, um técnico de manutenção e um técnico de receção, todos guardados na lista `lista_funcionarios`.

5.4 Gerente

Herda de: Funcionario

Principais atributos:

- `__bonus` (privado) — calculado automaticamente como 10% do salário

Principais métodos:

- `mostrar_informações(self)` — sobrescreve o método do Funcionario, mostrando salário + bónus
- `gerar_relatorio(self, funcionarios)` — percorre a lista de funcionários e mostra a informação de cada um

Como é utilizada no programa:

No menu de funcionário, a opção "3 - Gerar relatório de funcionários" chama `gerente.gerar_relatorio(lista_funcionarios)`, que percorre todos os funcionários (gerente, rececionista, técnico de manutenção e técnico de receção) e mostra os dados de cada um, aproveitando o polimorfismo do `mostrar_informações()`.

5.5 Recepcionista

Herda de: Funcionario

Principais atributos:

- `_turno` (propriedade, só aceita "manhã", "tarde" ou "noite")

Principais métodos:

- `mostrar_informações(self)` — sobrescreve o método, incluindo o turno
- `registrar_hospede(self, hospede, lista_hospedes)` — adiciona o hóspede à lista
- `listar_hospedes(self, lista_hospedes)` — percorre a lista e mostra cada hóspede

Como é utilizada no programa:

É usada no menu de hóspede (ao fazer reserva, `recepcionista.registrar_hospede(...)` adiciona o novo hóspede à lista) e no menu de funcionário (opção "1 - Listar hóspedes" chama `recepcionista.listar_hospedes(lista_hospedes)`).

5.6 TecnicoManutencao

Herda de: Funcionario

Principais atributos:

- `_especialidade` (propriedade, só de leitura)
- `_reparos` (lista, inicialmente vazia)

Principais métodos:

- `mostrar_informacoes(self)` — sobrescreve o método, incluindo a especialidade
- `registrar_reparo(self, descricao)` — adiciona uma reparação à lista `_reparos`
- `listar_reparos(self)` — percorre a lista e mostra todas as reparações

Como é utilizada no programa:

No menu de funcionário, a opção "4 - Registrar reparo (Técnico de Manutenção)" chama `tecnico_manutencao.registrar_reparo(...)` com a descrição introduzida pelo utilizador.

5.7 TecnicoRecepcao

Herda de: Recepcionista e TecnicoManutencao (herança múltipla)

Principais atributos:

- `_turno` (herdado/definido como em Recepcionista)
- `_especialidade` (definido como em TecnicoManutencao)
- `_reparos` (lista, definida no próprio `__init__`)

Principais métodos:

- `registrar_hospede(self, hospede, lista_hospedes)` — chama explicitamente `Recepcionista.registrar_hospede(self, ...)`
- `registrar_reparo(self, descricao)` — chama explicitamente `TecnicoManutencao.registrar_reparo(self, ...)`
- `mostrar_informacoes(self)` — sobrescrito, mostra nome, id, idade, salário, turno e especialidade (juntando informação das duas funções)

Como é utilizada no programa:

No `main.py` é criado `tecnico_recepcao = TecnicoRecepcao(...)`, e no menu de funcionário a opção "5 - Registrar reparo (Técnico de Recepção)" usa este objeto. É o exemplo de herança múltipla pedido no enunciado, juntando as capacidades de um rececionista e de um técnico de manutenção na mesma pessoa.

```
class TecnicoRecepcao(Recepcionista, TecnicoManutencao):
    def __init__(self, nome, id_func, idade, salario, turno, especialidade):
        Funcionario.__init__(self, nome, id_func, idade, salario)
        self.turno = turno
        self._especialidade = especialidade
        self._reparos = []
```

5.8 Quartos (classe abstrata), QuartoSimples e QuartoLuxo

Herda de: Quartos herda de ABC; QuartoSimples e QuartoLuxo herdam de Quartos

Principais atributos:

- numero, tipo, _ocupado
- preco_diaria (propriedade, com validação — não pode ser negativo)

Principais métodos:

- ocupar(self) / libertar(self) — alteram o estado _ocupado
- calcular_preco(self, dias) — método abstrato em Quartos; sobrescrito em cada subclasse
- mostrar_informações(self) — mostra número, tipo, preço e estado (Ocupado/Livre)

Como é utilizada no programa:

QuartoSimples tem preço base 50€/dia e calcula o preço como $\text{preco_diaria} * \text{dias}$. QuartoLuxo tem preço base 120€/dia e aplica um agravamento de 15% ($\text{preco_diaria} * \text{dias} * 1.15$, arredondado a 2 casas decimais). No main.py, lista_quartos é criada com 2 quartos simples (101, 102) e 2 quartos de luxo (201, 202).

```
class QuartoSimples(Quartos):
    def __init__(self, numero, preco_diaria=50):
        super().__init__(numero, "Simples", preco_diaria)
    def calcular_preco(self, dias):
        return self.preco_diaria * dias

class QuartoLuxo(Quartos):
    def __init__(self, numero, preco_diaria=120):
        super().__init__(numero, "Luxo", preco_diaria)
    def calcular_preco(self, dias):
        return round(self.preco_diaria * dias * 1.15, 2)
```

6. Herança

6.1 Herança simples

O projeto usa herança simples várias vezes. Os exemplos mais claros são:

- `Hospede(Pessoas)` — `Hospede` herda diretamente de `Pessoas`.
- `Funcionario(Pessoas)` — `Funcionario` também herda diretamente de `Pessoas`.
- `Gerente(Funcionario)`, `Recepcionista(Funcionario)` e `TecnicoManutencao(Funcionario)` — cada uma herda de uma única classe-mãe (`Funcionario`).
- `QuartoSimples(Quartos)` e `QuartoLuxo(Quartos)` — ambas herdam diretamente da classe abstrata `Quartos`.

6.2 Herança múltipla

O exemplo de herança múltipla do projeto é a classe `TecnicoRecepcao`, que herda ao mesmo tempo de `Recepcionista` e de `TecnicoManutencao`:

```
class TecnicoRecepcao(Recepcionista, TecnicoManutencao):
```

No construtor, em vez de usar `super().__init__()`, o código chama diretamente `Funcionario.__init__(self, ...)` para inicializar os dados base, e depois define manualmente `turno`, `_especialidade` e `_reparos`. Para evitar ambiguidades do MRO (ordem de resolução de métodos) do Python, a classe também chama explicitamente `Recepcionista.registrar_hospede(self, ...)` e `TecnicoManutencao.registrar_reparo(self, ...)` em vez de deixar o Python escolher automaticamente qual das duas classes-mãe usar. Isto confirma, tal como o enunciado sugeria, que existe herança múltipla entre `Recepcionista` e `TecnicoManutencao` através da classe `TecnicoRecepcao`.

7. Classe Abstrata

Uma classe abstrata é uma classe que não pode ser instanciada diretamente — serve apenas como "modelo" para outras classes, obrigando-as a implementar determinados métodos. Em Python, isto é feito com o módulo `abc`, usando `ABC` como classe-base e `@abstractmethod` para marcar os métodos que têm de ser sobrescritos.

No projeto existem duas classes abstratas:

- `Pessoas`, em `Pessoas/Pessoa.py` — herda de `ABC` e define `mostrar_informações()` como método abstrato. Nenhuma instância de `Pessoas` pode ser criada diretamente; só as suas subclasses (`Hospede`, `Funcionario` e as subclasses desta) podem ser usadas.
- `Quartos`, em `Quartos/Quartos.py` — também herda de `ABC` e define `calcular_preco(self, dias)` como método abstrato, obrigando `QuartoSimples` e `QuartoLuxo` a implementarem a sua própria forma de calcular o preço.

Isto corresponde exatamente ao que o enunciado pedia, que já apresentava "`Pessoa (abstrata)`" e "`Quarto (abstrato)`" na estrutura geral das classes.

8. Polimorfismo

Polimorfismo significa que objetos de classes diferentes podem responder de forma diferente à mesma chamada de método. No projeto, o exemplo mais evidente é o método `mostrar_informações()`, que está implementado de forma diferente em quase todas as classes:

- `Hospede.mostrar_informações()` — mostra nome, idade e quarto.
- `Funcionario.mostrar_informações()` — mostra nome, id, idade e salário.
- `Gerente.mostrar_informações()` — mostra nome, idade e salário + bônus.
- `Recepcionista.mostrar_informações()` — mostra também o turno.
- `TecnicoManutencao.mostrar_informações()` — mostra também a especialidade.
- `TecnicoRecepcao.mostrar_informações()` — mostra turno e especialidade ao mesmo tempo.

Isto é usado, por exemplo, no método `gerar_relatorio(self, funcionarios)` da classe `Gerente`, que percorre a `lista_funcionarios` e chama `funcionario.mostrar_informações()` para cada um, sem precisar de saber se é um `Gerente`, um `Recepcionista`, um `TecnicoManutencao` ou um `TecnicoRecepcao` — cada objeto "sabe" mostrar a sua própria informação.

Também existe polimorfismo no método `calcular_preco(dias)`, que `QuartoSimples` e `QuartoLuxo` implementam de forma diferente (um é apenas `preço × dias`, o outro aplica um agravamento de 15%).

9. Propriedades, Getters e Setters

O projeto usa `@property` e os respetivos `@x.setter` para controlar o acesso a vários atributos, validando os valores antes de os guardar. Alguns exemplos concretos:

- `Pessoas.nome` / `Pessoas.idade` — o setter de idade impede valores negativos, e o setter de nome impede nomes vazios.
- `Hospede.quarto` — o setter só permite atribuir o quarto se este não estiver ocupado (`if novo_quarto.ocupado: raise ValueError(...)`).
- `Recepcionista.turno` — o setter só aceita os valores "manhã", "tarde" ou "noite".
- `Quartos.preco_diaria` — o setter impede preços negativos.
- `Quartos.ocupado` e `TecnicoManutencao.especialidade` — propriedades só de leitura (sem setter), ou seja, só podem ser alteradas internamente (por exemplo, através dos métodos `ocupar()` e `libertar()`).

Estas propriedades protegem os dados da classe: em vez de se poder mudar um atributo diretamente para qualquer valor, o setter garante que só valores válidos são aceites, o que evita erros no programa (por exemplo, um quarto com preço negativo ou um hóspede com idade negativa).

10. Estruturas de Dados e Ciclos

O projeto usa principalmente listas como estrutura de dados:

- `lista_quartos` — lista com os objetos `QuartoSimples` e `QuartoLuxo` criados no início do `main.py`.
- `lista_hospedes` — lista (inicialmente vazia) onde os hóspedes são adicionados à medida que fazem check-in, através de `repcionista.registrar_hospede(...)`.
- `lista_funcionarios` — lista com o gerente, o rececionista, o técnico de manutenção e o técnico de receção.
- `_reparos` — lista usada dentro de `TecnicoManutencao` (e também de `TecnicoRecepcao`) para guardar as descrições das reparações registadas.

Quanto a ciclos, o programa usa sobretudo:

- `while` — para manter os menus (menu principal, menu de hóspede e menu de funcionário) em funcionamento até o utilizador escolher sair ou voltar.
- `for` — para percorrer `lista_quartos` (por exemplo, para mostrar os quartos livres ou todos os quartos), `lista_hospedes` (para listar hóspedes ou procurar um hóspede pelo nome) e `lista_funcionarios` (dentro de `gerar_relatorio`).

Estas estruturas e ciclos correspondem ao que o enunciado pedia, nomeadamente o uso de "estrutura de dados (por exemplo, listas, dicionários, tuplos, etc.) e laços de repetição (for, while, etc.)". No código entregue não foram encontrados dicionários nem tuplos — apenas listas.

11. Fluxograma

Foram fornecidas três imagens de fluxograma. As duas primeiras descrevem o fluxo de reserva/estadia/check-out — a segunda é mais completa, incluindo a entrada do gerente e a divisão entre recepção e manutenção nas reparações — e a terceira descreve especificamente o fluxo de um quarto de luxo (QuartoLuxo). As três imagens estavam legíveis, sem partes impercetíveis.

11.1 Fluxograma geral (reserva, estadia e check-out)

O fluxo começa em "Início" e segue para "Selecionar QuartoSimples". Surge então uma decisão — "Quarto está ocupado": se sim, volta a "Escolher outro quarto" e repete a seleção; se não, segue para "Definir número, tipo e preço diário", depois "Associar hóspede" e "Marcar como ocupado". Entra-se em "Realizar estadia", de onde surge a decisão "Há pedido de reparação?". Se sim, segue para "Registrar reparação" e "Executar manutenção", voltando à estadia. Se não, segue para "Calcular conta pelos dias", "Executar checkout", "Limpar associação", "Marcar como livre" e termina em "Fim".

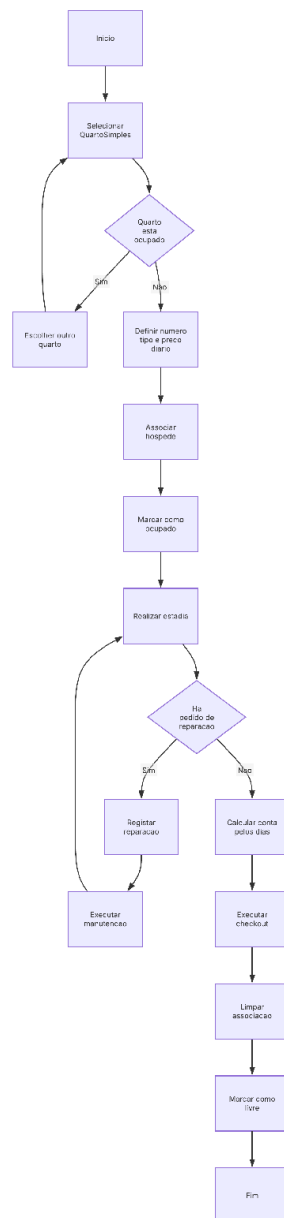


Figura 3 — Fluxograma geral: reserva, estadia e check-out

11.2 Fluxograma completo (com gerência e divisão recepção/manutenção)

Este fluxograma detalha melhor o processo: começa em "Início" com dois ramos paralelos — um para o pedido de reserva do hóspede e outro para o gerente gerar o relatório de funcionários (correspondendo a `gerente.gerar_relatorio()`). No ramo da reserva: "Hóspede pede reserva" → "Rececionista registra hóspede" → decisão "Há quarto livre?". Se não houver, segue para "Informar indisponibilidade" e termina. Se houver, segue para a decisão "Tipo de quarto" (Luxo ou Simples), depois "Escolher QuartoLuxo"/"Escolher QuartoSimples", "Associar quarto ao hóspede", "Marcar quarto como ocupado" e "Estadia". Durante a estadia há a decisão "É necessária reparação?": se sim, segue para "Técnico de recepção registra pedido" e depois "Técnico de manutenção repara", voltando à estadia; se não, segue para "Calcular conta", "Checkout", "Libertar quarto" e "Fim".

Este fluxograma mostra claramente, através do ramo "Técnico de recepção registra pedido" seguido de "Técnico de manutenção repara", a ligação entre as funções de recepção e de manutenção — o que no código corresponde à classe `TecnicoRecepcao`, que junta `registrar_hospede()` (de `Recepcionista`) com `registrar_reparo()` (de `TecnicoManutencao`).

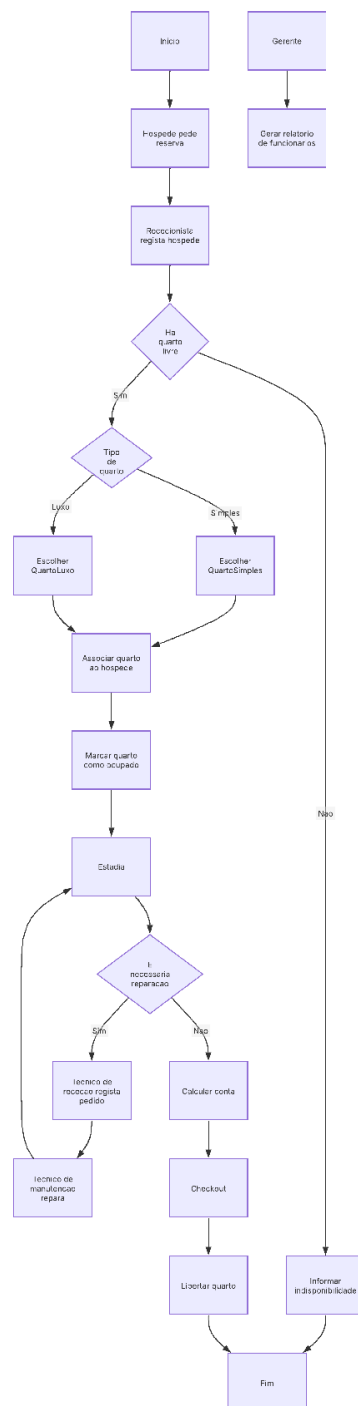


Figura 4 — Fluxograma: fluxo completo com recepção e gerência

11.3 Fluxograma do QuartoLuxo

Este terceiro fluxograma repete a mesma lógica do primeiro, mas aplicada especificamente ao QuartoLuxo: "Entrada" → "Selecionar QuartoLuxo" → decisão "Quarto ocupado?" (se sim, "Escolher outro quarto"; se não, "Definir preço diário premium") → "Associar ao hóspede" → "Marcar como ocupado" → "Estadia no quarto luxo" → decisão "Pedido de reparação?" (se sim, "Registrar reparação" e "Manutenção especializada", voltando à estadia; se não, "Calcular conta: dias × preço premium" → "Checkout" → "Limpar associação ao hóspede" → "Marcar como livre" → "Saída"). Este fluxo corresponde ao método QuartoLuxo.calcular_preco(dias), que aplica o preço premium com o agravamento de 15% em vez do cálculo simples do QuartoSimples.

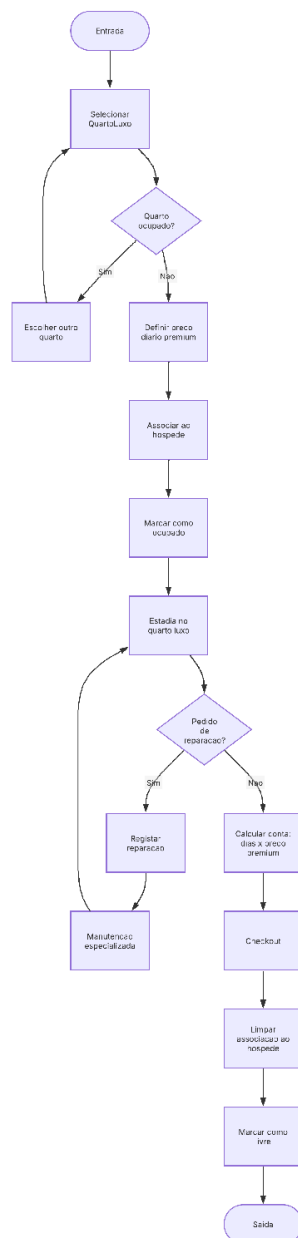


Figura 5 — Fluxograma: fluxo específico do QuartoLuxo

12. Funcionamento do Programa

Quando o programa arranca (main.py), são criados automaticamente:

- 4 quartos: 2 QuartoSimples (101, 102) e 2 QuartoLuxo (201, 202), guardados em lista_quartos;
- 4 funcionários: um Gerente (Carlos Sousa), um Recepcionista (Ana Silva, turno da manhã), um TecnicoManutencao (Rui Costa, especialidade Eletricidade) e um TecnicoRecepcao (Marta Reis, turno da tarde, especialidade Canalização), guardados em lista_funcionarios;
- uma lista_hospedes vazia, que vai ser preenchida ao longo da execução.

De seguida, é apresentado o "MENU" principal, dentro de um while que corre até o utilizador escolher "3 - Sair":

- Opção 1 — Entrar como hóspede: abre um submenu com 4 opções.
- Opção 2 — Entrar como Funcionário: abre outro submenu com 6 opções.
- Opção 3 — Sair: termina o programa.

12.1 Menu Hóspede

No submenu de hóspede:

- 1 - Fazer reserva: pede nome, idade e dias de estadia. Se a idade for inferior a 18 anos, mostra um erro e volta ao menu (continue). Caso contrário, é criado um Hospede, são mostrados os quartos livres (percorrendo lista_quartos com um for), o utilizador escolhe o número do quarto, e se esse quarto existir e estiver livre, é feita a atribuição (hospede.atribuir_quarto(quarto)) e o registo (recepcionista.registrar_hospede(hospede, lista_hospedes)).
- 2 - Ver conta: pede o nome do hóspede, procura-o em lista_hospedes (com um for) e, se encontrado, mostra o valor de hospede.calcular_conta().
- 3 - Fazer check-out: da mesma forma, procura o hóspede e chama hospede.checkout(), removendo-o depois de lista_hospedes.
- 4 - Voltar: sai do submenu.

12.2 Menu Funcionário

No submenu de funcionário:

- 1 - Listar hóspedes: chama recepcionista.listar_hospedes(lista_hospedes).
- 2 - Listar quartos: percorre lista_quartos e chama mostrar_informacoes() em cada quarto.
- 3 - Gerar relatório de funcionários: chama gerente.gerar_relatorio(lista_funcionarios), que mostra a informação de todos os funcionários (demonstrando o polimorfismo).
- 4 - Registrar reparo (Técnico de Manutenção): pede uma descrição e chama tecnico_manutencao.registrar_reparo(descricao).
- 5 - Registrar reparo (Técnico de Recepção): pede uma descrição e chama tecnico_recepcao.registrar_reparo(descricao).
- 6 - Voltar: sai do submenu.

Este funcionamento corresponde de perto ao primeiro fluxograma apresentado (secção 11.1 e 11.2): seleção/criação do quarto, associação ao hóspede, estadia, decisão sobre reparações e, por fim, cálculo da conta e checkout.

13. Testes Realizados

O ficheiro `main.py` não contém testes automáticos (não existem blocos de teste, nem ficheiros de teste separados, nem resultados de execução gravados). O que existe é a criação inicial de objetos concretos, que funciona como uma espécie de "teste manual" de arranque do sistema:

- Criação de 4 quartos com números e tipos diferentes (`QuartoSimples 101` e `102`; `QuartoLuxo 201` e `202`) — testa, na prática, se os construtores de `QuartoSimples` e `QuartoLuxo` funcionam e se herdam corretamente de `Quartos`.
- Criação de 4 funcionários diferentes (`Gerente`, `Recepcionista`, `TecnicoManutencao` e `TecnicoRecepcao`) — testa, na prática, se todas as subclasses de `Funcionario` se conseguem construir corretamente, incluindo o caso de herança múltipla do `TecnicoRecepcao`.

Para além disto, o próprio fluxo do menu (fazer reserva, ver conta, check-out, listar hóspedes/quartos, gerar relatório, registar reparos) funciona como teste manual quando o programa é corrido e usado interativamente. Como não foi fornecido nenhum registo de execução (output/print do programa a correr), não é possível confirmar aqui resultados concretos de execução — apenas que o código está preparado para estes cenários.

14. Dificuldades Encontradas

Não foi fornecida nenhuma informação direta sobre as dificuldades sentidas durante o desenvolvimento deste projeto em concreto, pelo que esta secção apresenta apenas dificuldades técnicas que são plausíveis quando se implementa este tipo de sistema, sem afirmar que ocorreram de facto:

- Gerir corretamente a herança múltipla do `TecnicoRecepcao`, nomeadamente decidir chamar os métodos das classes-mãe de forma explícita (`Recepcionista.registrar_hospede(self, ...)` e `TecnicoManutencao.registrar_reparo(self, ...)`) para evitar ambiguidades do MRO do Python.
- Garantir que a atribuição de um quarto a um hóspede só acontece quando o quarto está mesmo livre, validando isso no setter da propriedade `quarto`.
- Coordenar as várias listas (`lista_quartos`, `lista_hospedes`, `lista_funcionarios`) ao longo de um menu com vários submenus em ciclos `while` aninhados.

15. Conceitos de Programação Aplicados

- Programação Orientada a Objetos — todo o sistema é modelado através de classes e objetos (pessoas, funcionários, hóspedes, quartos).
- Classes e objetos — por exemplo, Hospede, Gerente, QuartoLuxo, e as respectivas instâncias criadas em main.py.
- Encapsulamento — atributos protegidos (`_salario`, `_id_func`) e privados (`__quarto`, `__dias_estadia`, `__bonus`), acedidos através de propriedades.
- Herança simples — `Hospede(Pessoas)`, `Funcionario(Pessoas)`, `Gerente/Recepcionista/TecnicoManutencao(Funcionario)`, `QuartoSimples/QuartoLuxo(Quartos)`.
- Herança múltipla — `TecnicoRecepcao(Recepcionista, TecnicoManutencao)`.
- Polimorfismo — `mostrar_informações()` e `calcular_preco()` implementados de forma diferente em cada subclasse.
- Classes abstratas — `Pessoas` e `Quartos`, ambas usando `ABC` e `@abstractmethod`.
- Módulos — código organizado em pastas (`Pessoas`, `Pessoas/Hospedes`, `Pessoas/Funcionarios`, `Quartos`) e ficheiros importados em main.py.
- Estruturas de dados — listas (`lista_quartos`, `lista_hospedes`, `lista_funcionarios`, `_reparos`).
- Ciclos — `while` (menus) e `for` (percorrer listas).

16. Conclusão

O trabalho desenvolvido cumpre os objetivos principais definidos no enunciado: foi construído, em Python, um sistema de gestão de hotel que permite gerir quartos de diferentes tipos e reservas feitas por hóspedes, usando herança simples, herança múltipla, polimorfismo, classes abstratas e módulos, com recurso a listas e a ciclos for/while.

A estrutura de classes implementada corresponde, de forma muito próxima, à estrutura geral apresentada no enunciado (Pessoa abstrata → Hóspede/Funcionário → Gerente/Rececionista/TecnicoManutencao → TecnicoRecepcao; e Quarto abstrato → QuartoSimples/QuartoLuxo), com alguns pequenos ajustes de implementação (por exemplo, nomes de classes como Pessoas em vez de Pessoa, ou TecnicoRecepcao a herdar de Rececionista e TecnicoManutencao em vez de apenas repetir os seus métodos).

Com este projeto foi possível perceber, na prática, como a herança (simples e múltipla), o polimorfismo e as classes abstratas ajudam a organizar um sistema com várias entidades parecidas mas com comportamentos diferentes, evitando repetir código e tornando mais fácil adicionar novos tipos de quartos ou de funcionários no futuro.

17. Verificação Final

Requisitos cumpridos:

- Sistema em Python para gerir quartos (simples e luxo) e reservas de hóspedes.
- Herança simples (Hospede, Funcionario e subclasses, QuartoSimples/QuartoLuxo).
- Herança múltipla (TecnicoRecepcao, herdando de Recepcionista e TecnicoManutencao).
- Polimorfismo (mostrar_informacoes() e calcular_preco()).
- Classes abstratas (Pessoas e Quartos, com ABC e @abstractmethod).
- Uso de módulos (código dividido em várias pastas/ficheiros).
- Uso de estruturas de dados — listas — e de ciclos for/while.

Requisitos parcialmente cumpridos ou com diferenças face ao enunciado:

- O enunciado sugeria o nome da classe "Pessoa"; no código, a classe chama-se Pessoas.
- O enunciado sugeria que TecnicoRecepcao reunisse os métodos de Recepcionista e TecnicoManutencao; no código entregue, isso é feito através de herança múltipla direta (TecnicoRecepcao(Recepcionista, TecnicoManutencao)), com chamadas explícitas às classes-mãe — está de acordo com o pedido, apenas com uma abordagem de implementação ligeiramente diferente da sugestão do enunciado (que indicava "de Recepcionista" / "de TecnicoManutencao" como se fossem apenas métodos próprios).
- O enunciado sugeria o uso de estruturas de dados como "listas, dicionários, tuplos, etc."; o projeto usa apenas listas, não usando dicionários nem tuplos.
- Não foram encontrados testes automáticos nem registos de execução no código entregue, pelo que a secção de testes se baseia apenas na análise do código.

Inconsistências encontradas entre enunciado, código e fluxograma:

- Não foram encontradas inconsistências graves. A principal diferença é apenas de nomenclatura (Pessoa vs. Pessoas) e a forma concreta de implementar a herança múltipla do TecnicoRecepcao, já referidas acima.
- O fluxograma mais detalhado (secção 11.2) inclui um ramo de "Informar indisponibilidade" quando não há quarto livre, que não está explicitamente escrito como mensagem no código (no main.py, nesse caso, é apenas mostrado "Quarto inexistente ou ocupado!"). A lógica é semelhante, mas vale a pena confirmar se o texto da mensagem deve ser ajustado.

Partes a rever antes de entregar:

- Preencher na capa o ano letivo e os números dos alunos, caso sejam pedidos pela escola.
- Confirmar se o nome da classe Pessoas deve passar a Pessoa, conforme o enunciado, ou se a equipa prefere manter o nome atual.
- Confirmar se é suposto existirem dicionários ou tuplos no projeto, já que o enunciado os refere como exemplo (mas não como obrigatório).